

Commands for building Docker images and running containers

- Docker build
 - Builds the image for a docker container
 - Docker build -t container-name:tag . - builds an image with a tag associated with it
- Docker run
 - deploys the container using the image
 - -p tag maps that your application is using in the docker container to the port on your host computer
 - Example: docker run -p host-port-number:container-port-number container-name:<optional-tag>

Useful command for listing docker images with their tags:

1. Cd into you directory where dockerfile is stored
2. Run docker ps -a to see a list of docker images and their tags
 - a. docker image ls is another way to see images
 - b. docker ps will let you see running containers
3. docker run is the basic command for running a docker container
 - a. Add a -d to the command to run it in the background, you'll be able to run it and still do stuff in the command terminal you run it in

Common Docker Statements to be Used in Dockerfiles

1. COPY - copy files/directories from one directory to another
2. FROM - pulls down images from to use as a template for your custom
3. WORKDIR - sets the directory for the application you are trying to run
4. RUN - a command you want to run during the build process for the docker container
5. CMD - specifies the default command that will be run when the container is running

Things to keep in mind about Docker

- Docker was made to make managing your dependencies, packages, operating systems, and everything that is needed to run your application easier
 - Without it you would have to manage all of those things manually and run into issues like not having the correct packages downloaded, not having the right version of the operating system, the services, config files, and user dependencies are all set up correctly
 - Containers run isolated from the operating system, so it should not depend at all on anything running in your computer
 - Containers should also not affect each other as well
 - Great alternative to virtual machines since it is much lighter in terms of storage

- Images are the instructions for how containers run, containers are the processes that are actually running
- Docker does have a caching mechanism when it comes to images, using layers
 - Layers are created with each command written in a dockerfile
 - Can be considered image diffs
 - Each change creates a new layer
 - Docker stores all individual layers
 - Layers are immutable, so deleting a layer is really just creating a new layer with whatever you wanted to be removed gone
 - Each layer is cached, so when you run a docker container it saves the result of each command and just re-uses it if it wasn't changed
- Tags are mutable, meaning they can be updated to point to a different image
- The -e tags will allow you pass environment variables to your container when you need it

How to Dockerize a Spring Boot Application

1. Open up IntelliJ
2. Go to the Maven window(if you don't see it, go to View, click on it for the dropdown, then click Tools, then select Maven, and it should appear)
3. You should see the name of your project and underneath it should be a folder named Lifecycle, click on the dropdown arrow and you'll see a list of programs to choose from
4. Click on the 'clean' program to remove outdated version of project
5. Then click on the 'compile' program to re-compile the Spring Boot project
6. Then click on 'install' to package the project into a jar file and have it copied to your maven directory (.m2)
7. Then in your dockerfile make sure to add a statement that copies the jar file that will be in the target directory of the project into the working directory for the docker:
 - a. COPY target/boxing-fighter-management-app-0.0.1-SNAPSHOT.jar /app/fighter-management-app.jar

How to Dockerize a React Application

1. Open up VSCode, and create a Dockerfile
2. Move the Dockerfile out of the src directory and into the main project directory
3. In the Dockerfile, write a FROM statement that pulls node:20-apline image and set it as builder
 - a. AS builder statement basically makes sure that the outcome of the commands and statements you run before pulling down a new image gets saved and that you can use the product of those commands in the next stage, otherwise, the outcome would be lost

4. Set working directory inside of the container(WORKDIR /app)
5. Make sure to copy the package*.json to use docker's caching mechanism
 - a. COPY package*.json ./
 - b. This copies over the package files that details what dependencies the react project needs and uses
6. Once that's done, run a npm ci(RUN npm ci)
 - a. This installs all of the dependencies that are detailed in the package-json.file that you copied over
7. Also, create a .dockerignore file and add the node-modules directory to it
 - a. That's where all of the external dependencies are stored, so if you copy it over to the working directory in docker container, it will be a huge memory and time sink
8. After that, COPY everything in the project directory for the react project into the working directory for the docker container(COPY . . - the first dot is referencing current directory you're in, which should be the project directory, the second dot represents the working directory of the docker container)
9. Then create a build of the react application so that it creates web assets that will be used by the browser (RUN npm build)
 - a. can't just serve react code because the browser will not know what to do with that
10. After that pull the nginx image (FROM nginx:1.25-apline) to your container
 - a. Nginx is a better performing web server that will serve the web assets to browsers
11. Remove the default welcome pages from the nginx server (rm -rf /usr/share/nginx/html/*)
12. Copy the web assets from the build into the html directory for nginx (COPY --from=builder /app/dist/ /usr/share/nginx/html/*)
13. Write an EXPOSE <port-number> command to let users know what port the react container is being mapped to
14. Run CMD ["nginx", "-g", "daemon-off;"] to run the nginx server after everything else has been completed.
15. For the nginx server you'll to create your own .conf file to make sure that it looks that the index.html file that was created in React
 - a. First create a nginx.conf file in the root project directory
 - b. Then in that file, here's you'll need to add these things:
 - i. server {...} - this is where you set config properties for the server
 1. root /usr/share/nginx/html; - tells the server where to look for the physical files of your website
 2. index index.html; - makes sure if the root url (/) is visited it automatically serves the index.html webpage

3. `try_files $uri $uri/ /index.html;` - tells nginx to check for the index files in these specific places in this specific order. If it can't find it in the first too, it will use the index.html that's last in the order
 - a. `$uri`-looks for the file in the current directory
 - b. `$uri/`-looks for the file in a directory in the current directory
 - c. `/index.html`-just serves the index.html file
- c. Once the file's created, copy the custom nginx.conf file to the `/etc/nginx/conf.d/default.conf` file (COPY `nginx.conf` `/etc/nginx/conf.d/default.conf`)
 - i. Run this command before copying the build assets over to the html directory for nginx